

Memoria P3

Ejercicio 1) Explicación y justificación del diseño de la solución proporcionada:

Fichero de problema: En primer lugar, los términos Enano, Hobbit, Madera, Mineral, Mithril, Especia y Alimento deben ser constantes, y no tipos independientes. Por ello, en el estado inicial, hemos indicado los tipos de los personajes mediante el predicado `personajeEs` y las capacidades de extracción de cada tipo mediante `puedeExtraerTipo`. Esto facilita posteriores modificaciones de capacidades de extracción sin tener que cambiar la jerarquía de tipos del dominio.

Por otro lado, se indica que disponemos de tres personajes iniciales, de los cuales `Enano2` no está disponible. Para indicar esto, hemos usado la Hipótesis del Mundo Cerrado y, por ello, al no escribir el predicado disponible `Enano2`, el traductor de Fast Downward asume que es falso.

En relación al mapa de la Tierra Media, hemos decidido indicar ambos sentidos entre cada par de nodos adyacentes, mediante hechos del tipo `(camino X Y)` y `(camino Y X)`. Esto garantiza la bidireccionalidad del movimiento de los personajes por el mapa. Además, en esta versión final se ha incorporado ya desde el ejercicio 1 el tratamiento especial del camino Rivendell-Moria mediante el predicado `caminoDestruible`, de forma que, si se cruza en cualquiera de los dos sentidos, el camino queda eliminado en ambas direcciones y no puede volver a utilizarse.

Finalmente, para el objetivo, en vez de forzar qué personaje debe extraer cada recurso o en qué localización, establecemos que el objetivo sea simplemente que el Mithril y el Alimento se estén generando. De esta forma, el planificador decide automáticamente qué personaje disponible y qué nodo de recurso son más adecuados.

Fichero de dominio: En primer lugar, para evitar errores provenientes de declarar las especies de los personajes, como Enano o Hobbit, como tipos en la sección `:objects`, hemos diseñado un tipo abstracto llamado `TipoPersonaje` y hemos definido las palabras Enano y Hobbit como constantes globales pertenecientes a dicho tipo. Mediante el predicado `(personajeEs ?p - Personaje ?t - TipoPersonaje)`, indicamos el tipo de cada personaje.

Por otro lado, en la acción `ExtraerRecurso`, como no se puede pasar la especie del personaje para saber qué puede extraer un personaje en concreto, la solución ha sido activar el requerimiento de precondiciones existenciales. Gracias a esto, la acción `ExtraerRecurso` es capaz de realizar una búsqueda entre las especies existentes mediante el cuantificador `exists`.

Para asegurar que los personajes se comporten de acuerdo a las reglas del mundo físico planteado, hemos introducido el predicado `(disponible ?p)`. De esta manera

garantizamos que un personaje ocupado en tareas de extracción no pueda realizar simultáneamente otras acciones. Por su parte, en la acción ExtraerRecurso, el efecto incluye (not (disponible ?p)) para que el personaje se quede trabajando allí.

Fichero sas_plan: Al ejecutar el planificador con seq-opt-lmcut, el plan obtenido tiene longitud 11 y coste 11, ya que en este ejercicio todas las acciones tienen coste unitario. El plan mueve a Enano1 desde Tharbad hasta Bree, luego hasta Rivendell y finalmente cruza hacia Moria para extraer Mithril. Por otro lado, Hobbit1 viaja desde Lothlorien hasta Hobbiton siguiendo la ruta Lothlorien, Amon Hen, Fangorn, Isengard, Helm's Deep, Tharbad y Hobbiton, donde extrae Alimento. La diferencia con versiones anteriores se debe a que ahora el camino Rivendell-Moria se destruye al cruzarse. Por tanto, el planificador evita que Hobbit1 utilice dicho camino antes que Enano1, ya que eso impediría posteriormente que Enano1 llegase a Moria por la ruta óptima. Enano2 no aparece en el plan porque no está disponible inicialmente, ya que se encuentra ocupado en una misión diplomática.

Fichero output.sas: Contiene el problema PDDL traducido a una representación SAS+ finita y explícita, lista para la búsqueda. Incluye las variables de estado y sus valores posibles, el estado inicial, el objetivo, y las acciones ya instanciadas con sus precondiciones, efectos y coste.

Podemos observar que se generan 8 variables principales: la disponibilidad de Hobbit1, la localización de Hobbit1, si se está generando Alimento, la disponibilidad de Enano1, la localización de Enano1, si se está generando Mithril, y el estado de existencia de los caminos bidireccionales entre Moria y Rivendell. Esto muestra que Fast Downward simplifica el problema y conserva solo la información relevante para alcanzar el objetivo.

En el estado inicial del fichero observamos que Hobbit1 está en Lothlorien, Enano1 está en Tharbad, ambos están disponibles, los caminos entre Moria y Rivendell están activos, y todavía no se están generando ni Alimento ni Mithril. El objetivo traducido consiste precisamente en que las variables correspondientes a generando(alimento) y generando(mithril) pasen a ser verdaderas.

Respecto a las instancias del operador extraerrecurso, Fast Downward genera únicamente 7 acciones posibles:

```
extraerrecurso enano1 erebor mineral
extraerrecurso enano1 fangorn maderas
extraerrecurso enano1 lothlorien maderas
extraerrecurso enano1 mirkwood maderas
extraerrecurso enano1 moria mineral
extraerrecurso enano1 moria mithril
extraerrecurso hobbit1 hobbiton alimento
```

Estas instancias son coherentes con la definición del problema. Enano1 puede extraer Mineral, Mithril y Madera, por lo que aparecen acciones para los nodos de esos recursos. Hobbit1 solo puede extraer Alimento, por lo que solo aparece la acción correspondiente a Hobbiton. No aparece ninguna instancia de extracción para Enano2, ya que no está disponible inicialmente y, por tanto, no puede ejecutar acciones.

Ejercicio 2) Explicación y justificación del diseño de la solución proporcionada:

Fichero de problema: Para este ejercicio, hemos mantenido el mapa de la Tierra Media del ejercicio anterior con sus mismas localizaciones y caminos, incluyendo el tratamiento especial del camino destruible entre Rivendell y Moria. Esta decisión se mantiene de forma incremental, ya que dicho camino forma parte de la lógica general del mapa.

Hemos definido y situado a los personajes según el enunciado, dejando a Enano2 como no disponible por su misión diplomática. En lugar de crear tipos en PDDL para cada raza, hemos decidido representar las clases Hobbit, Mago y Enano como constantes mediante el predicado personajeEs, manteniendo la estructura del ejercicio anterior donde todos los individuos son objetos de tipo Personaje.

Respecto a los objetos, hemos ubicado el Anillo en Rivendell y la Espada en Lothlorien. No hemos colocado el ChalecoMithril en el estado inicial porque se crea dinámicamente con la acción materializarChaleco. Además, hemos vinculado Orodruin como lugar de destrucción del Anillo usando lugarDestruccion, para evitar hardcodear la localización en el dominio. Nuestro objetivo final es simplemente activar anilloDestruido.

Fichero de dominio: Hemos construido el dominio de forma incremental, conservando Viajar y ExtraerRecurso, e incorporando las nuevas acciones requeridas. Hemos modificado Viajar añadiendo la precondition negativa (not (miembroComunidad ?p)) para impedir que un personaje se mueva solo si ya pertenece a la Comunidad.

Para formarComunidad, hemos diseñado la acción para que un hobbit y un mago disponibles se unan en una misma localización, dejando que el planificador elija quiénes y dónde se reúnen para optimizar el plan. Una vez juntos, hemos empleado el predicado comunidadEn para representar la localización conjunta de la Comunidad. Esta decisión simplifica mucho el estado, ya que evita tener que actualizar de forma redundante la posición individual de todos sus miembros.

Hemos unificado la recogida del Anillo, la Espada y el Chaleco en la acción recogerObjeto, diferenciándolos con predicados de clase, como esAnillo, esEspada y esChaleco. Hemos establecido que solo un hobbit de la Comunidad pueda recoger el Anillo, y que la Espada y el Chaleco solo los pueda recoger ese mismo portador.

Para el equipamiento, hemos programado materializarChaleco para que un mago de la Comunidad lo cree en una localización donde se esté generando Mithril. Finalmente, destruirAnillo exige que el portador sea un hobbit de la Comunidad, tenga los tres objetos y esté en el lugar marcado mediante lugarDestruccion.

Nota de diseño: Aunque en una versión anterior habíamos usado predicados dobles como comunidadFormada y comunidadNoFormada, hemos optado por sustituirlos por precondiciones negativas como (not (comunidadFormada)). Así hemos logrado un dominio más limpio sin tener que actualizar manualmente predicados opuestos.

Fichero sas_plan: Hemos obtenido un plan de longitud y coste 14. El planificador muestra que la ruta óptima consiste en extraer primero Mithril con Enano1 en Moria, mover a Hobbit4 desde Bree hasta Rivendell, formar allí la Comunidad con Mago1, recoger el Anillo, viajar juntos hasta Moria para materializar y recoger el ChalecoMithril, pasar por Lothlorien para recoger la Espada y avanzar finalmente hasta Orodruin para destruir el Anillo. El plan es coherente con el diseño porque deja al planificador escoger automáticamente a Hobbit4 y Mago1, que son los personajes más favorables por su ubicación inicial.

Ejercicio 3) Explicación y justificación del diseño de la solución proporcionada:

Fichero de problema: Hemos desarrollado este ejercicio como una continuación directa del ejercicio 2, por lo que hemos mantenido la representación general del mapa, los recursos, los objetos y las acciones básicas para recoger objetos y destruir el Anillo. La diferencia principal consiste en estudiar cómo afecta al planificador el aumento progresivo del tamaño de la Comunidad. Para ello hemos generado cuatro versiones del dominio y del problema: 3a, 3b, 3c y 3d.

En el fichero de problema de todos los apartados hemos partido del mismo conjunto de personajes: cuatro hobbits, dos magos, un elfo y dos enanos, todos inicialmente disponibles. Hemos conservado intactas las posiciones iniciales, los caminos, incluyendo el camino destruible Rivendell-Moria, los recursos y la ubicación del Anillo y de la Espada. Además, hemos seguido utilizando el predicado lugarDestruccion para marcar Orodruin como el destino del Anillo. El objetivo final sigue siendo activar anilloDestruido.

Fichero de dominio: La modificación esencial afecta a las acciones formarComunidad y viajarComunidad, ya que la Comunidad va creciendo en cada apartado.

A pesar de este crecimiento, una vez formada la Comunidad, no actualizamos la posición individual de cada personaje. En su lugar, hemos empleado el predicado global comunidadEn para representar su ubicación conjunta. Hemos optado por esto porque los miembros deben desplazarse siempre juntos, por lo que mantener

posiciones individuales generaría información redundante y aumentaría innecesariamente el espacio de búsqueda.

Por otra parte, hemos mantenido la precondition negativa (not (miembroComunidad ?p)) en la acción Viajar para impedir que un integrante se mueva individualmente una vez dentro del grupo. Las acciones materializarChaleco, recogerObjeto y destruirAnillo se conservan con la misma lógica: el portador debe ser un hobbit de la Comunidad que consiga el Anillo, el Chaleco de Mithril, creado por el mago donde se extrae Mithril, y la Espada antes de llegar a Orodruin.

Nota sobre simetrías: Al introducir varios hobbits equivalentes, hemos detectado una gran fuente de simetrías que ralentiza la búsqueda. Para reducir este problema, en el apartado 3d hemos introducido el predicado idMenor para imponer un orden entre los hobbits elegidos, evitando que el planificador evalúe la misma Comunidad en distinto orden de parámetros.

Tamaño de la Comunidad	Tiempo de Planificación ("Planner time")	Longitud del Plan encontrado ("Plan length")
(a): 1 hobbit y 1 mago	0.29s	14
(b): 2 hobbits y 1 mago	0.54s	16
(c): 3 hobbits y 1 mago	1.26s	17
(d): 3 hobbits, 1 mago y 1 elfo	978.44s	23

Discusión de los resultados obtenidos y análisis de la clase de complejidad computacional de la planificación automática: Los experimentos muestran un crecimiento muy claro del tiempo de planificación a medida que el grupo se amplía. Sin embargo, la longitud de los planes crece de forma mucho más moderada: pasa de 14 acciones en el apartado 3a a 23 acciones en el apartado 3d. Esto indica que la dificultad principal no está solo en que el plan final sea más largo, sino en el aumento del espacio de búsqueda que el planificador debe explorar para demostrar la optimalidad del resultado.

Para profundizar en la clase de complejidad computacional de la planificación automática y en cómo escala la resolución de problemas se ha seguido la bibliografía recomendada. **(Se ha usado IA como ayuda para buscar información en la bibliografía).**

En primer lugar, como señalan Russell & Norvig (2022) en el Capítulo 11 lo que hace, el planificador es traducir la especificación PDDL a una representación interna (SAS+) y evaluar sistemáticamente las combinaciones de acciones aplicables mediante búsqueda heurística hacia delante (heuristic forward search). Además, el

uso del alias --alias seq-opt-lmcut , hace que se deba explorar el espacio de estados para comprobar que el plan elegido es óptimo, y esto, como viene en la presentación de la práctica, es muy costoso.

Ghallab, Nau y Traverso (2016) en "Automated Planning and Acting" argumentan que, como el problema de determinar si existe un plan válido en entornos STRIPS proposicionales es PSPACE-completo, implica que no existen garantías de escalabilidad. Esto quiere decir que variaciones aparentemente mínimas en la definición de las acciones o en los objetos utilizados pueden alterar radicalmente el espacio de búsqueda.

Para entender lo que sucede en el apartado (d) usamos el modelo teórico de Geffner y Bonet (2013) ("A Concise Introduction to Models and Methods for Automated Planning"), que muestra que el número de acciones totalmente instanciadas que el planificador debe evaluar internamente crece de forma polinómica respecto al número de objetos, donde el exponente es el número de parámetros de la acción. Por ejemplo, una acción con 4 parámetros y 100 objetos candidatos genera 100^4 instancias posibles. En el dominio del apartado 3d, la acción formarComunidad pasa a requerir 5 parámetros de tipo Personaje más una Localizacion , lo que provoca que haya una cantidad enorme de tuplas candidatas.

¿El planificador proporcionaría con rapidez un plan si nuestra Comunidad estuviese formada por 4 hobbits, 1 mago, 1 elfo, 1 enano y 2 humanos? No, el planificador no proporcionaría un plan con rapidez. Hemos visto que con 5 miembros (apartado 3d) el tiempo de ejecución explota pasando de 1.26 segundos a 978, por lo que elevar los miembros a 9 personajes llevará a una multiplicación combinatoria de los parámetros de formarComunidad y viajarComunidad que no es sostenible. Una posible solución sería no usar el mismo alias óptimo y usar otros que aunque no te garanticen encontrar el plan más corto, si te permite encontrar soluciones factibles en un tiempo razonable

¿Es posible que, al igual que en la Práctica 2, surjan soluciones simétricas?
¿De qué modo podrían aparecer estas y qué impacto tendrían en el rendimiento? Sí, es posible que surjan simetrías, ya que disponemos de múltiples instancias del mismo tipo, como Hobbit1, Hobbit2 y Hobbit3 inicialmente situados en Hobbiton. Si la acción formarComunidad se codifica requiriendo simplemente varias variables de tipo Personaje que cumplan el predicado personajeEs ?h Hobbit, el planificador puede evaluar de manera independiente permutaciones equivalentes de los mismos individuos. Por ejemplo, una Comunidad formada por Hobbit1, Hobbit2 y Hobbit3 es semánticamente la misma que otra formada por Hobbit2, Hobbit1 y Hobbit3, pero para el planificador pueden ser asignaciones distintas de parámetros.

Esto tiene un impacto importante en el rendimiento, ya que el planificador dedica tiempo a explorar ramas redundantes que no aportan soluciones cualitativamente

distintas. Para mitigar este problema en el apartado 3d, definimos un orden entre las instancias mediante el predicado `idMenor`. Posteriormente, en las precondiciones de la acción `formarComunidad`, forzamos a que las variables se asignen respetando dicho orden creciente.

Ejercicio 4) Explicación y justificación del diseño de la solución proporcionada:

Fichero de problema: Para este ejercicio, hemos partido del escenario base del ejercicio 3a, manteniendo la Comunidad reducida de 1 hobbit y 1 mago y la ruta general hacia Orodruin. La ampliación principal ha consistido en incorporar las nuevas facciones, es decir, humanos, orcos y corsarios, necesarias para la creación del Uruk-Hai. Siguiendo la filosofía de diseño previa, hemos modelado estas razas como constantes de `TipoPersonaje` asignadas a objetos de tipo `Personaje` mediante el predicado `personajeEs`.

En cuanto a la disponibilidad inicial, no incluimos como disponible al humano de Edoras y, por la Hipótesis del Mundo Cerrado, se toma como no disponible. Esto obliga al planificador a movilizar al humano de Bree para las tareas de construcción. En el estado inicial, hemos definido que la Torre de Hechicería necesita Mineral y Madera, mientras que el Extractor requiere Madera. Además, hemos etiquetado tanto la Torre de Hechicería como el Extractor como edificios malignos, de modo que los recursos necesarios para construirlos no pueden haber sido extraídos por enanos. Finalmente, hemos marcado el Mineral con el predicado `requiereExtractor` y hemos ampliado el objetivo final: ahora exige destruir el Anillo, activar `urukHaiCreado` y garantizar la presencia final de un humano en Bree.

Fichero de dominio: Hemos construido este dominio expandiendo el código previo e importando requerimientos avanzados como `:universal-preconditions`, `:existential-preconditions`, `:disjunctive-preconditions` y `:adl`. Hemos modificado la acción `ExtraerRecurso` con una condición que distingue si un recurso requiere o no un Extractor. En concreto, si el recurso requiere Extractor, solo puede extraerse si hay uno construido en la misma localización.

Hemos diseñado la acción `Construir` sin pasar el recurso como parámetro de la acción. En su lugar, la propia acción consulta qué recursos necesita cada edificio mediante el predicado `recursoNecesario`. Para ello se ha implementado un forall con una implicación `imply`: para todo recurso, si ese recurso es necesario para el edificio, entonces debe existir algún personaje trabajando y generándolo. Además, si el edificio es maligno, se exige que el trabajador que generó ese recurso no sea Enano. De esta forma, el planificador descarta automáticamente el uso de recursos extraídos por enanos para tareas malignas.

A diferencia de la acción `ExtraerRecurso`, en `Construir` no se elimina la disponibilidad del personaje constructor, porque el enunciado indica que la

construcción de edificios es una excepción y que el constructor vuelve a estar disponible tras la acción.

Para modelar la logística de la Especia, hemos creado `RecogerEspecia`, obligando a que un corsario recoja físicamente la Especia en el lugar donde otro corsario la está extrayendo. Después, ese mismo corsario debe transportarla mediante `Viajar` hasta `Isengard`. Finalmente, la acción `CrearUrukHai` exige que haya una Torre de Hechicería construida, que un corsario con Especia esté en esa misma localización y que el mago creador no pertenezca a la Comunidad.

Fichero `sas_plan`: Hemos obtenido un plan de coste 45 ejecutado bajo el alias `seq-sat-lama-2011` con límite de tiempo. Este alias prioriza encontrar un plan válido y no necesariamente óptimo.

En el plan obtenido se combinan varias líneas de acción. Por un lado, la Comunidad formada por `Hobbit4` y `Mago1` recoge el Anillo, consigue el Chaleco de Mithril en `Moria`, recoge la Espada en `Lothlorien` y finalmente destruye el Anillo en `Orodruin`. Por otro lado, `Enano1` se desplaza hasta `Moria` para extraer Mithril, que permite materializar el chaleco. Para la parte industrial, `Orco3` extrae Madera en `Lothlorien`, `Humano2` viaja desde `Bree` hasta `Moria` para construir el Extractor y `Orco2` extrae Mineral en `Moria`. Después, `Humano2` regresa a `Bree` para satisfacer el tercer objetivo del problema. En paralelo, `Corsario1` extrae Especia en `Tolfolas` y `Corsario2` la recoge y la transporta físicamente hasta `Isengard`. Finalmente, `Mago2` construye la Torre de Hechicería en `Isengard` y crea el Uruk-Hai cuando la Especia ya se encuentra allí.

Ejercicio 5) Explicación y justificación del diseño de la solución proporcionada:

Este ejercicio mantiene el escenario del Ejercicio 1 (mismo estado inicial y objetivo de generar Mithril y Alimento), pero modifica la evaluación de los desplazamientos introduciendo costes no unitarios. Debido a las limitaciones de `Fast Downward` para gestionar funciones numéricas complejas dependientes de parámetros en búsquedas óptimas, hemos optado por una estratificación de acciones en el dominio (`Viajar`, `ViajarCoste2`, `ViajarCoste3`, `ViajarCoste5`, `ViajarCoste8` y `ViajarCoste3Destruible`), activando el requerimiento `:action-costs` para ir incrementando la función (`total-cost`).

En el fichero del problema, se inicializó explícitamente (`= (total-cost) 0`) y se estableció la directiva (`:metric minimize (total-cost)`), forzando al planificador a buscar la optimización económica del plan en lugar de la longitud de pasos.

Compare el plan obtenido en este ejercicio y en el Ejercicio 1: En el ejercicio 1, todas las acciones tienen coste unitario, por lo que el planificador minimiza el número total de acciones. En cambio, en el ejercicio 5, el planificador minimiza el coste total acumulado en `total-cost`.

En el ejercicio 1, el plan obtenido tiene coste 11 y longitud 11. Enano1 viaja de Tharbad a Bree, de Bree a Rivendell y de Rivendell a Moria, donde extrae Mithril. Hobbit1 viaja desde Lothlorien hasta Hobbiton pasando por Amon Hen, Fangorn, Isengard, Helm's Deep y Tharbad, y después extrae Alimento.

En el ejercicio 5, el plan obtenido tiene coste 23. Enano1 mantiene la ruta Tharbad, Bree, Rivendell y Moria, ya que necesita llegar a Moria para extraer Mithril y el coste de cruzar Rivendell-Moria es 3. Sin embargo, Hobbit1 modifica su ruta respecto al ejercicio 1: desde Lothlorien va a Amon Hen y después continúa por Dead Marshes, Minas Morgul, Minas Tirith, Edoras, Helm's Deep, Tharbad y Hobbiton. Aunque esta ruta tiene más desplazamientos que una ruta directa por Fangorn e Isengard, evita utilizar el tramo Fangorn-Amon Hen, que tiene coste 8.

Uso de IA generativa

- LLM utilizado: Gemini
- Propósito: Principalmente se ha utilizado para generar código repetitivo, como la parte del mapa en el :init, para ayudar a entender los archivos sas_plan y output.sas, como ayuda para formatear la memoria y los comentarios, y como ayuda en momentos en los que me quedaba atascado y no se me ocurría cómo seguir avanzando.
- Ejemplos de Prompt + Respuesta:

Prompt: tengo esta accion en unarchivo de dominio de pddl, que solo puedo pasarle estos parametros:

(:action ExtraerRecurso

:parameters (?p - Personaje ?l - Localizacion ?r - Recurso)

sin embargo tengo que comprobar si ese personaje puede extraer ese recurso especifico, ya que cada personaje tiene asociado un tipopersonaje que a su vez tiene asociado que recursos puede extraer

Respuesta: Para solucionar esto sin tocar los parámetros de la acción, se me ocurre que utilices precondiciones existenciales (:existential-preconditions). La lógica es la siguiente: en lugar de "aplanar" los datos en el estado inicial, vamos a crear un "puente" lógico de tres niveles en el dominio (Personaje $\rightarrow$ TipoPersonaje $\rightarrow$ Recurso) y dejaremos que el planificador busque la relación usando un cuantificador exists.

Análisis crítico: Con esta solución propuesta, efectivamente en la acción ExtraerRecursos ya puedo determinar si el tipo del personaje puede extraer el recurso r sin necesidad de ningún parámetro extra en la acción.

—

Prompt: puedes resumirme este texto manteniendo el mismo tono?

(Aquí le paso el texto que tenía originalmente pero por motivos de espacio no lo incluyo)

Respuesta: Me ha dado directamente el texto resumido, que incluyo en la justificación del ejercicio 2.

Análisis crítico: Después de revisar el texto proporcionado, observo que se ha conseguido el objetivo de reducir el tamaño del mismo, sin modificar demasiado el tono, más bien recortando “la paja”, conectores, etc.

—

Prompt: Tengo este archivo que se supone que tiene que tardar en ejecutarse unos 500-600 segundos, pero pasa de 1000, estoy ya desesperado y no se que puede ser. Intuyo que tiene que ser por las simetrías o que el espacio de búsqueda es enorme, pero no se como se puede solucionar. Ayuda

Respuesta: Qué pasa, tío! Qué buena pregunta. El motivo por el que tu ejecución se queda colgada son las simetrías masivas en los parámetros de la Comunidad. En el caso 3d, tu acción viajarComunidad tiene como parámetros a tres hobbits a la vez: (?h1 ?h2 ?h3 ?m ?e - Personaje ...). Como para el planificador Hobbit1, Hobbit2 y Hobbit3 son objetos idénticos del mismo tipo, cada vez que la comunidad intenta dar un paso del mapa de la localización X a la Y, el planificador evalúa todas las permutaciones posibles de orden de los parámetros ($3! = 6$ combinaciones idénticas). Multiplica eso por los 21 nodos del mapa y el árbol de búsqueda explota de forma exponencial (clásico problema PSPACE-completo). El truco maestro: Romper la simetría ordenando los objetos. Al obligar a que las variables de los parámetros de la comunidad cumplan estrictamente que $?h1 < ?h2 < ?h3$, le prohíbes a Fast Downward evaluar combinaciones espejo redundantes (como evaluar el viaje con Hobbit2 Hobbit1 Hobbit3). El factor de ramificación se reduce de forma drástica y el alias obligatorio seq-opt-lmcut encontrará el plan óptimo original en cuestión de milisegundos.

Análisis crítico: Tras analizar la respuesta, me cuadra porque además es similar a lo que hemos hecho en la práctica de 2, de establecer un orden para romper las simetrías, y además en el ejercicio 1 se indica que de ser necesario se pueden incluir más predicados. Tras ejecutarlo efectivamente se reduce drásticamente el tiempo de ejecución.